Treating the Tremors We Built into Robots

What a checked execution layer can promise

Mike Helsel
Working draft for technical review; AI assistance disclosed

18 October 2026

Abstract

A smooth reference, an accepted request, and a completed task are different guarantees. We formalize part of their connection for a scalar execution layer built from an established four-stage moving average. Lean checks a rational finite history against jerk and acceleration limits at every future real time. A finite candidate rule preserves committed motion and can delay or reject a new execution. We prove that completed event tails can be removed without changing future motion, and that a checker using adjacent knot cells has the same acceleration decisions. A separate, conditional theorem carries the physical state from time zero and proves position tracking, progress, and position arrival for one return task.

A predeclared numerical comparison starts three methods from the same reference and physical state. The retained filter has about 0.40 times the squared-jerk integral of a native planner with matched arrival, but about 2.09 times its extra forward travel. Only the native fast method completes the tighter task. These results support a checked-contract case study, not a new smoothing algorithm, a general VLA remedy, or a physical robot result.

1 The guarantee has three parts

A planner requests a task goal g. An executor produces a reference q. A controller moves the physical position x. Their errors satisfy

$$x - g = (x - q) + (q - g). \tag{1}$$

A small tracking error $x - q$ can coexist with a failed task. The reference may arrive too late, travel too far after a reversal, or continue an old goal after rejecting a request.

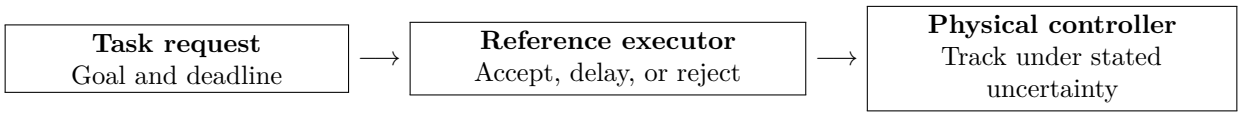


Figure 1. Each boundary needs its own contract. A reference limit does not become a physical limit merely because the controller receives that reference.

The case study asks whether a finite retained history can make these contracts explicit and executable. It does not infer whether a new request is noise or a real change of intent. The title describes an engineering symptom; it does not propose a neurological mechanism.

Prior work sets the scope. FIR planning and derivative limits are established. Biagiotti and Melchiorri already discuss acceleration growth under an overlapping reversal and waiting as a response [1]. Their later work addresses time optimization under kinematic and frequency constraints [2]. We use the reversal as a regression case, not a discovery. Ruckig supplies online jerk-limited motion from nonzero initial states [3]. Reference governors and ModelPlex already connect future constraints, uncertainty, and checked execution [4, 5]. The proposed contribution is the specialized Lean contract and its reproducible boundaries. Its research value still needs external specialist review.

2 Exact finite checks of a retained reference

Let $h > 0$ be one box width, and write $z_+ = \max(z, 0)$. The step response of four cascaded box averages is

$$F(z) = \frac{1}{24} \sum_{k=0}^{4} (-1)^k \binom{4}{k} (z-k)_+^4.$$

For a finite list of execution times and signed increments $E = ((\tau_i, \delta_i))_{i=1}^n$, define

$$q_E(t) = g_0 + \sum_i \delta_i F((t - \tau_i)/h).$$

The reference is C^3. Its velocity is piecewise cubic, acceleration piecewise quadratic, and jerk piecewise affine. Signed cancellation remains in the reference; we do not replace it by a sum of absolute event magnitudes.

At current time c, let S contain c and every future knot $\tau_i + kh$, $k = 0, \ldots, 4$. In each nonempty adjacent cell of S, add the interior zero of jerk when jerk is nonconstant and has such a zero. Call the resulting set C. When jerk is constant, acceleration is affine; its endpoints suffice. After the final knot all derivatives vanish.

Theorem 1 (Exact reference checks)**.** *For any finite event list, $h > 0$, $J \geq 0$, and $A \geq 0$,*

$$\big(\forall s \in S,\ |q_E'''(s)| \leq J\big) \iff \big(\forall t \geq c,\ |q_E'''(t)| \leq J\big),$$

$$\big(\forall s \in C,\ |q_E''(s)| \leq A\big) \iff \big(\forall t \geq c,\ |q_E''(t)| \leq A\big).$$

For rational inputs, the Lean Boolean checks have these all-future real contracts. Invalid widths and negative limits are rejected.

Proof structure. An affine function has its absolute maximum at a cell endpoint. A quadratic function can also attain its maximum magnitude where its derivative vanishes. The formal argument constructs cells from the event list, including the current time and the final tail. Repeated, unsorted, expired, and queued events are covered. It does not rely on a sampled graph.

The first acceleration implementation enumerates candidate pairs. A proved replacement takes the minimum strictly greater knot for each left endpoint. The theorem `same_acceptance` equates the two Boolean results, including invalid-input rejection. This optimization changes evaluation work, not the accepted set.

Theorem 2 (Removal of completed tails)**.** *Remove every event with $\tau_i + 4h \leq c$, and add the sum of its increments to g_0. For every $t \geq c$, the resulting position, velocity, acceleration, and jerk equal those of the original reference. Both guard decisions are preserved.*

Completed responses have position contribution δ_i and zero derivative. This result does not preserve past motion or bound the number of active events. An unbounded arrival rate can still require unbounded active history.

3 Admission can repair a limit and still fail a task

A new increment is relative to the final committed target. An append-only admission rule retains earlier events. Execution cannot precede admission. The separately proved causal wrapper also orders execution times. Rejection retains the existing reference. A new event contributes zero through its execution time, so the executed position and its first three derivatives remain unchanged.

For a finite supplied list of candidate execution times, the timed selector checks $\tau \geq c$, jerk, acceleration, and completion of every event by a reference deadline D. A selected candidate therefore preserves the prefix, meets those reference bounds, and reaches the final committed target at rest by D. A result of `none` says only that the supplied candidates failed. The selector is not a complete trajectory planner. Its current general interface has no velocity guard; the concrete return below has a separate velocity bound.

Fixed reversal. Use $h = .2$ s. A $+20$ mm event starts at .22 s. A return to zero becomes available at .52 s. An immediate -20 mm event passes the jerk limit 7.5 m/s^3, but its acceleration reaches $21/32 = .65625$ m/s^2, above

the .5 limit. Trying execution times $[.52, .62]$ s selects .62 for deadline 1.70 s. The retained reference then has $|q'| \le 1/15$ m/s, $|q''| \le .5$ m/s^2, and $|q'''| \le 7.5$ m/s^3. It reaches zero with zero derivatives at 1.42 s. The same candidate list returns `none` for deadline 1.30 s.

These are exact rational Lean fixtures. They show a useful distinction: repairing a reference limit requires time, and a deadline may not allow it. Waiting while retaining an old motion is not an emergency stop policy.

4 A conditional physical guarantee

The scalar plant and feedback are

$$\dot{x} = v, \qquad \dot{v} = a, \qquad \dot{a} = r(t)[u + w(t) - a], \tag{2}$$

$$u = q'' - 8\sigma^2 e_x - 6\sigma e_v - \tfrac{1}{2} e_a + \frac{5}{48\sigma} q''', \tag{3}$$

where $e_x = x - q$, $e_v = v - q'$, and $e_a = a - q''$. The rate satisfies $8\sigma \le r(t) \le 12\sigma$, and $|w(t)| \le W$. Here u, w, a have units m/s^2 and σ, r have units s^{-1}.

Define the transformed errors

$$z_1 = e_x, \qquad z_2 = e_x + \frac{e_v}{4\sigma}, \qquad z_3 = e_x + \frac{e_v}{2\sigma} + \frac{e_a}{16\sigma^2}.$$

Theorem 3 (Tracking and one position task)**.** *Let $H, R, W, J \ge 0$ and $\sigma > 0$. Suppose a classical physical trajectory of (2)–(3) exists on $[0, H]$ and satisfies the stated pointwise uncertainty bounds. If $|z_i(0)| \le R$, $|q'''| \le J$ on that interval, and*

$$W + \frac{J}{48\sigma} \le 2\sigma^2 R,$$

then $|e_x| \le R$, $|e_v| \le 8\sigma R$, and $|e_a| \le 64\sigma^2 R$ throughout it.

For the delayed-return history above, take $H = 2.5$ s, $\sigma = 10$ s^{-1}, $W = .01$ m/s^2, $R = .2$ mm, and $J = 7.5$ m/s^3. Under the same initial and trajectory premises,

$$|x(t) - q(t)| \le .2 \text{ mm} \quad (0 \le t \le 2.5), \qquad x(.5) - x(0) \ge 1 \text{ mm},$$

$$|x(t)| \le .2 \text{ mm} \quad (1.42 \le t \le 2.5).$$

The proof derives the reference derivatives and jerk premise from the finite history, then uses the invariant error cube. For progress, $q(.5) = 779/250000$ m and $q(0) = 0$; the two position-error bounds suffice. The sole physical initial condition is at time zero. The state is not reset when a new request arrives.

The final claim is position arrival, not physical rest. It does not imply the physical acceleration or jerk equals a reference limit. The existence premise is material: we have not proved existence for arbitrary discontinuous inputs. The numerical parameter values are illustrative, not identified from a robot. Separate terminal-settling results are outside this paper's composed task theorem.

5 Comparison from the same moving state

The experiment starts all methods from zero and uses the same FIR reference through the left side of .52 s. At that revision, the common reference state is

$$(q, q', q'') = (77/19200,\ 23/480,\ 5/16)$$

in SI units. The physical state is carried from time zero and is identical across methods for each plant case.

We compare retained FIR, native fast Ruckig, and native Ruckig with minimum duration .90 s. The latter matches FIR's 1.42 s reference arrival within the predeclared 10^{-12} s tolerance. The binary native duration is not an exact rational identity. The native initial state is rounded once to binary floating point. All methods use the same physical feedback and reference limits $V = .1$ m/s, $A = .5$ m/s^2, $J = 7.5$ m/s^3.

The frozen grid has rates $80, 100, 120$ s^{-1}, disturbances $0, .01$ m/s^2, three methods, two deadlines, and output intervals 1 and .5 ms: 72 traces. A full task requires request acceptance, initial progress, reference conformity, tracking, and all three physical terminal bands from the deadline through 2.5 s:

$$|x| \le .2 \text{ mm}, \quad |v| \le .001 \text{ m/s}, \quad |a| \le .02 \text{ m/s}^2.$$

A rejected revision fails the task even if the later position happens to be near zero. Numerical terminal checks include event sides but are not an all-time proof.

Fine-grid results, six plant cases	Retained FIR	Native fast	Native matched
Full task, deadline 1.70 s	6/6	6/6	6/6
Full task, deadline 1.30 s	0/6	6/6	0/6
$\int_{.52}^{2.5} (\dot{a})^2\, dt$ (m^2/s^5)	6.067–6.121	17.307–17.494	15.171–15.250
Extra forward excursion (mm)	14.315–14.331	6.851–6.872	6.851–6.872
Suffix path length (mm)	32.650–32.669	17.734–17.763	17.723–17.770
Command total variation (m/s^2)	1.973–1.978	2.771–2.881	2.610–2.780

Motion-quality ranges use the generous task over $[.52, 2.5]$ s. Extra forward excursion is measured beyond the physical position at revision, not beyond the final target. The squared-jerk integral is a smoothness proxy, not motor energy. Command variation includes jumps.

An independent replay used 80-digit arithmetic and a different physical propagator. It checked all 271,884 state rows and 1,356 event rows with no task-decision mismatch. Fresh native plans and 7,922 native queries matched the saved exports. Both output grids give the same decisions. The largest position difference between propagators was 4.2×10^{-16} m. This is agreement between numerical models, not a hardware error bar.

The FIR/native-matched squared-jerk ratio is .398–.402, while the forward-excursion ratio is 2.085–2.090. Native fast alone passes the tight task. Matching arrival removes one confound; it does not match the paths or make the native method a squared-jerk optimizer. A stronger performance claim would need a smooth baseline with the same arrival and an explicit excursion limit. We do not infer general superiority from this comparison.

6 Proof artifact and practical limits

The portable package binds 50 local source modules to three frozen proof gates. Its replay builds local sources into an empty directory, checks every local declaration for unsafe code and unapproved axioms, runs separate kernel checks for nine main modules, and requires twelve false clients to fail for mathematical reasons. Public dependency caches remain a declared trust boundary. The permitted axioms are `propext`, `Classical.choice`, and `Quot.sound`. The package wrapper itself awaits independent review.

Paper claim	Lean entry point
Exact future jerk	`check_real_iff`
Exact future acceleration	`checkAcceleration_real_iff`
Same successor acceptance	`SuccessorCheck.same_acceptance`
Remove completed tails	`HistoryPruning.future_jet_eq`
Timed finite candidate contract	`chosen_reference_contract`
Position task from initial physical state	`delayed_return_physical_position`

Exploratory shared-host timings put the new 32-event checker near 220 ms, versus about 22.5 s for the earlier enumerating check on the same fixed inputs. These are Lean `-run` measurements, not worst-case bounds or production native timings. They do not establish a 1 kHz implementation. Removing completed tails helps storage only when active event arrivals are also controlled.

The next practical test needs measured clocks, an execution schedule, a command interface, and a coupled robot model. Franka is the intended later platform; no robot was used here. Contact, payload variation, joint coupling, actuator saturation, and sampled actuation need their own assumptions and acceptance tests. The result does not certify a machine-safety performance level.

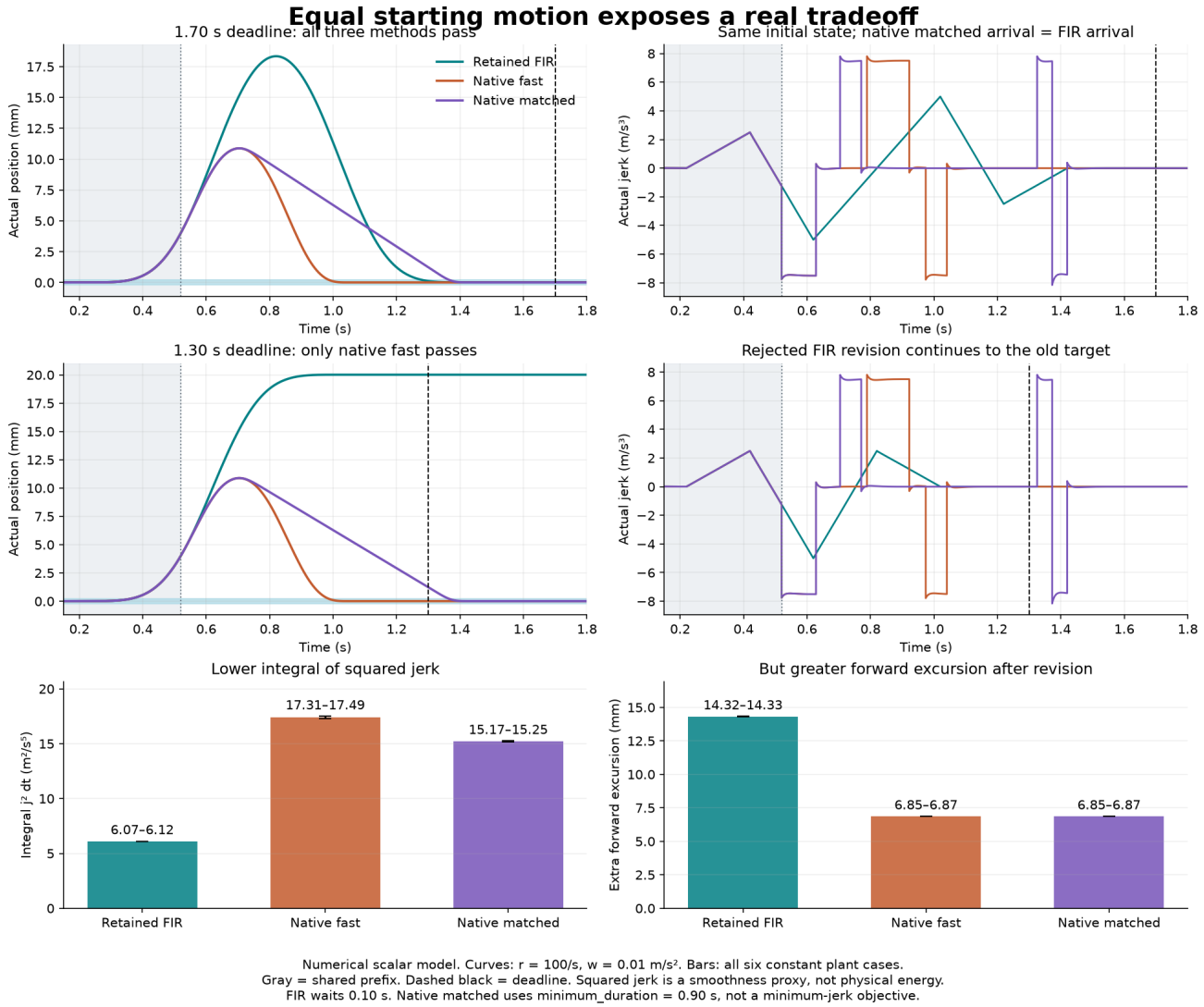


Figure 2: The accepted common-prefix comparison shows both smoothness and travel. Curves use the plant case stated in the figure. Bars integrate from .52 to 2.5 s; ranges cover all six plant cases. These are numerical results for the declared scalar model.

Relation to learned and industrial execution. Real-Time Chunking and VLA-RAIL already retain or combine actions across changing plans [6, 7]. The present scalar history is not a full action chunk. Industrial systems also expose path and timing choices: FANUC documents timing configuration, while Yaskawa describes its SMOOTH function [8, 9]. Those public interfaces motivate explicit execution contracts. They do not establish the vendors' internal algorithms or provide a measured baseline in this study.

Review status and authorship. The proof gates and numerical comparison have independent agent checks. No external human review, physical validation, public release, or paper submission has occurred. Mike Helsel set the research question, proposed the execution-layer direction, identified the timing interpretation, and required fair comparisons and independent challenge. Codex agents assisted with source review, mathematics, Lean code, numerical code, critique, figures, and writing. Affiliation, release scope, license, and venue remain owner decisions.

References

[1] L. Biagiotti and C. Melchiorri. FIR Filters for Online Trajectory Planning with Time- and Frequency-Domain Specifications. *Control Engineering Practice* 20(12), 1385–1399, 2012. https://doi.org/10.1016/j.conengprac.2012.08.005.

[2] L. Biagiotti and C. Melchiorri. Trajectory generation via FIR filters: A procedure for time-optimization under kinematic and frequency constraints. 2019. https://iris.unimore.it/handle/11380/1175126.

[3] L. Berscheid and T. Kröger. Jerk-limited Real-time Trajectory Generation with Arbitrary Target States. RSS, 2021. https://arxiv.org/abs/2105.04830v2.

[4] E. Garone, S. Di Cairano, and I. Kolmanovsky. Reference and command governors for systems with constraints: A survey on theory and applications. *Automatica* 75, 306–328, 2017. https://doi.org/10.1016/j.automatica.2016.08.013.

[5] S. Mitsch and A. Platzer. ModelPlex: Verified Runtime Validation of Verified Cyber-Physical System Models. Runtime Verification, 199–214, 2014. https://symbolaris.com/pub/modelplex.pdf.

[6] K. Black, M. Y. Galliker, and S. Levine. Real-Time Execution of Action Chunking Flow Policies. NeurIPS, 2025. https://arxiv.org/abs/2506.07339v2.

[7] Y. Zhao et al. VLA-RAIL: A Real-Time Asynchronous Inference Linker for VLA Models and Robots. 2025. https://arxiv.org/abs/2512.24673v1.

[8] FANUC Corporation. How to Configure 1kHz Mode. ROS 2 Driver Documentation v2.2.0. https://fanuc-corporation.github.io/fanuc_driver_doc/v2.2.0/docs/fanuc_driver/how_to_configure_1khz_mode.html.

[9] Yaskawa Motoman. SMOOTH Function Overview. 2023. https://knowledge.motoman.com/hc/en-us/articles/16313283895191-SMOOTH-Function-Overview.